Validate safely.
Deliver with evidence.

A Python/FastAPI transaction-validation API with gated automated delivery to Google Kubernetes Engine.

> **Business boundary**
> Synthetic transactions only. No money transfer, database or real payment-card data.

> **Verified**
> Automatic publication and rollout.
> Failed-test and security release blocks.
> Controlled rollback and monitoring alerts.

Representative successful run: normal SCM **#39** → **delivery #3**.
Live application version: **0.4.0**.

Public repository · Live API documentation · Registry

Strict input boundaries; pure transaction rules

- **GET /health**: liveness, separate from business validation.
- **POST /validate**: five required string fields; unknown fields rejected.
- Decimal money, two fractional digits; no float intermediates or silent rounding.
- Positive amount, configured limit, supported currency and distinct accounts.

FastAPI/Pydantic replace Spring Boot; uv/pytest replace Maven/JUnit. The same API/testing objectives are demonstrated.

```
Accepted → HTTP 200
{"valid":true,"errors":[]}

Zero amount → HTTP 200
{"valid":false,
 "errors":["AMOUNT_NOT_POSITIVE"]}

Numeric amount instead of string
→ HTTP 422, safe error metadata
```

These are contract examples. The evidence records actual successful, rejected and malformed-request checks.

A real cloud deployment—not a directory scaffold

2 nodes · 2 ready Pods

- Fixed Singapore GKE footprint; AMD64 application image.
- Non-root UID 10001, read-only root, dropped capabilities and resource limits.
- Restricted namespace admission, probes and independent runtime-image verification.
- Public API responds at **35.198.197.186**.

Pod recovery

Earlier approved Pod deletion produced a replacement and two available replicas **16.1373 seconds** later; 21/21 health samples passed.

Historical observation, not an SLA or node-failure test.

HTTP is unencrypted. Use synthetic data only. Trial resources require manual credit monitoring. No automated spending cap or teardown is claimed.

Normal SCM #39 → delivery #3: SUCCESS

1,349 tests passed

```
GitHub main → SCM polling
→ locked install / lint / tests
→ source + dependency gates
→ AMD64 build / image gate
→ native SUCCESS upstream trigger
→ independent archive verification
→ publish / registry readback
→ image-only rollout / smoke
```

> **Exact image; no rebuild**
>
> The transferred gated archive becomes the registry image. Raw manifest/config bytes and runtime Pod image identities are verified.

Delivery #3: 104.780 seconds total. Two owned ready Pods, public readiness and four smoke cases passed.

Controls: fixed approved controller code, isolated release worker, scoped credentials, serialized execution, resume disabled and original-candidate checks before writes.

Native causes, checksums and deployment reports

Failed tests and vulnerable images cannot proceed

Deliberate native Replay	Observed result	Release boundary
Source #37: failing assertion	1 failed / 1,348 passed	No package/image artifacts. No new delivery run.
Source #38: isolated pinned Debian base	**38 HIGH + 2 CRITICAL** 40 blocking findings	Image withheld. Never published or deployed.
Normal clean-main #39	All tests and gates passed	Automatically triggered successful delivery #3.

Faults were injected only in isolated Replay workspaces. Main and job definitions were not changed by the faults. These two demonstrations were operator-triggered, not SCM runs.

LOW Bandit findings remain visible. No known image findings in a database snapshot does not mean vulnerability freedom. No gate was weakened to obtain success.

Keep the failures; prove the recovery

- **Delivery #1:** stopped before credentials on an empty-variable shell error; corrected.
- **Delivery #2:** published and patched 0.4.0, then failed a public HTTP smoke request. Result remains FAILURE.
- Read-only inspection then verified 0.4.0, two ready Pods and all smoke cases.

Explicit controlled rollback

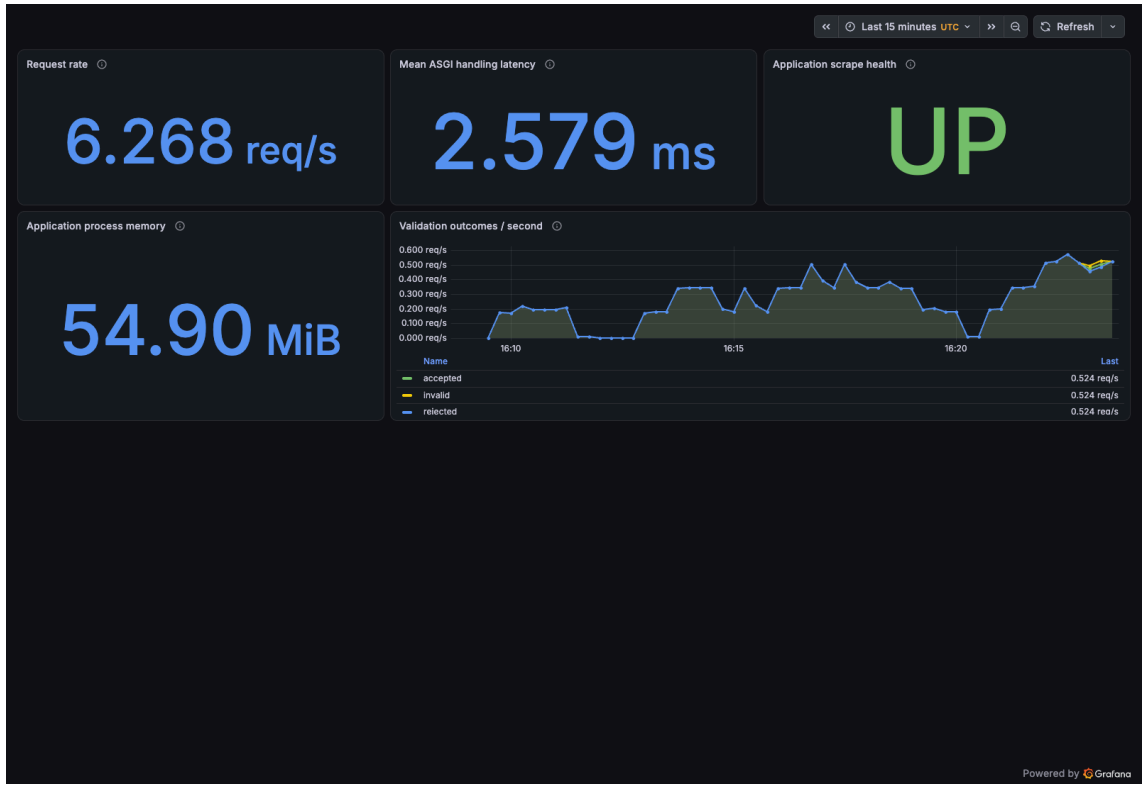
One conditional image-only PATCH restored known-good **0.3.0** in **33.43 seconds**.

Approved spec restored; two ready Pods, public version and four smoke cases verified.

A fresh revision subsequently completed #39 → #3 and restored 0.4.0. No uncertain PATCH or old tag was replayed.

Rollback was an explicit operator demonstration, not automatic error handling. Public readiness observations now precede single-attempt smoke checks. The precise earlier HTTP exception cause was not retained.

Cloud application; locally hosted monitoring



Actual screenshot, not a mock. This samples the load-balanced endpoint: counters can alternate between Pods. Do not treat it as reliable aggregate throughput or an SLO.

Grafana / Prometheus: request metrics, ASGI latency, scrape health, process memory and validation outcomes.

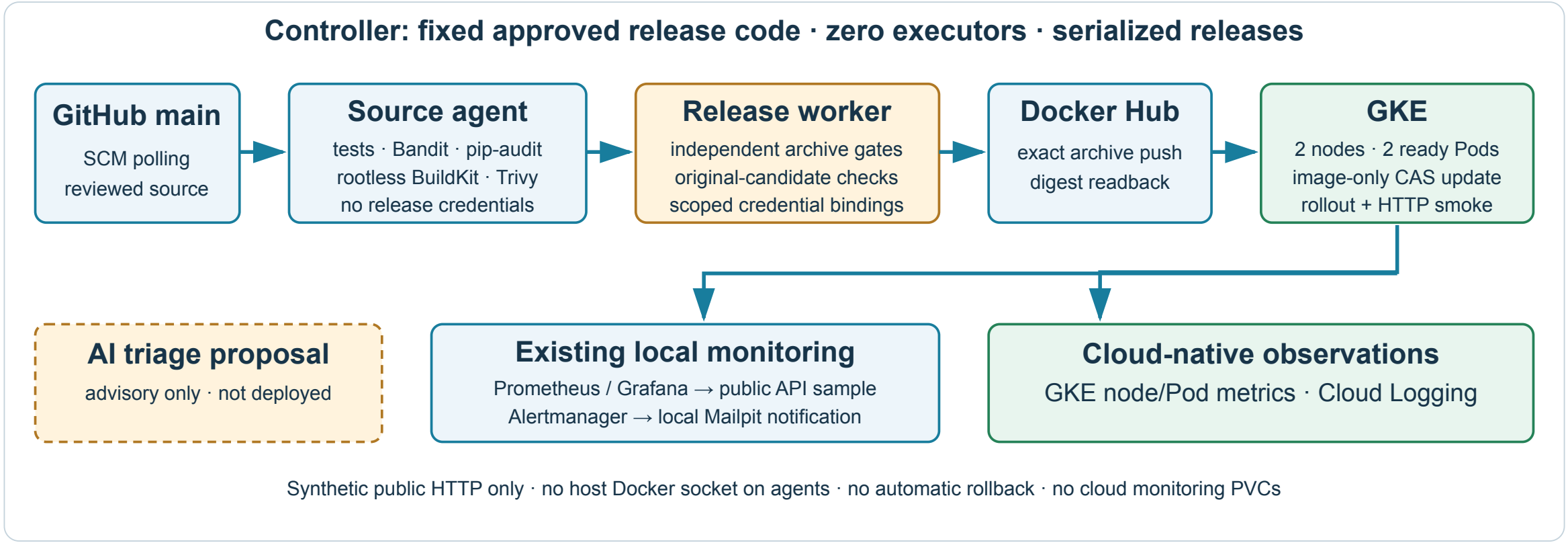
Native GKE metrics: actual CPU/memory for both nodes and both application Pods.

Cloud Logging: actual structured accepted/rejected/invalid events, with private resource labels omitted from evidence.

> **Actual notification and recovery**
> A deliberate scrape-port failure produced FIRING and RESOLVED Mailpit emails. Port 80 was restored; API health samples remained successful.

Monitoring-path failure, not induced API downtime. Mailpit proves local SMTP, not external email. No cloud monitoring volumes were created.

Separate source execution from release authority



Controller: zero executors. Source agent: no release credentials or host Docker socket. Release worker: exclusive executor, reviewed tools and scoped file bindings. AI is not in the runtime control path.

Advisory incident triage, not autonomous remediation

Operators navigate separate alerts, metrics, logs and deployment records. AI could assemble a cited timeline and identify missing evidence.

> Sanitized alert + approved evidence
> → bounded model input
> → schema and citation validation
> → human-reviewed summary

No secrets, arbitrary tools or mutation capability. Separate facts from hypotheses; missing diagnostics mean the cause remains unknown.

- Expected benefit: less navigation time and clearer handoffs; **not measured**.
- Risks: hallucinations, prompt injection, disclosure, stale evidence and cost escalation.
- Compare with human/runbook and deterministic-template baselines.
- Provider, budget, retention and access still require decisions.

> No AI model is deployed. The proposal and authored example are not implementation or measured results.

Detailed AI proposal and evaluation approach

Demonstrated capabilities; explicit limitations

Demonstrated	Evidence
Validation, tests and cloud health	Strict API; normal SCM #39; live 0.4.0 and two ready Pods
Automated delivery and release blocks	#39 → delivery #3; real test failure #37; vulnerability block #38
Recovery and rollback	Historical Pod recreation; explicit 0.4.0 → 0.3.0 rollback; fresh successful release
Metrics, logs and notification	Actual dashboard, native GKE resources, Cloud Logging and firing/resolved mail

Not claimed: production readiness, external SMTP, complete per-Pod Prometheus discovery, automatic rollback/spending cap, full live cancellation/restart fault coverage or deployed AI.

Public API and repository are evaluator entry points. Jenkins/Grafana/Mailpit are loopback-only. Keep credentials and private cloud identifiers out of the recording.